

Pharmacy Management System — Business Requirements Document

Document status: Phase 1 minimum scope baseline

Purpose: Define only the minimum connected pharmacy workflow the team must complete for Phase 1.

Introduction

Business Problem

The pharmacy needs one connected and reliable system for the operational flow from supplier purchasing and batch receipt through inventory, POS sales, invoicing, payments, returns/refunds, and reporting. These activities share stock, expiry, acquisition-cost, payment, and balance data; if they are handled as disconnected workflows, the pharmacy cannot reliably preserve FEFO allocation, prevent invalid stock changes, or trace financial and inventory effects back to their source transactions. Phase 1 therefore addresses the need for a single authoritative workflow built on the repository's existing Django foundation and shared Neon/PostgreSQL database.

Business Objectives

- Deliver the minimum complete supplier-to-report pharmacy workflow defined by this BRD within the approved Phase 1 scope.
 - Maintain accurate batch-aware inventory, block expired-stock sales, and allocate sale quantities using FEFO.
 - Preserve traceable inventory and financial history through source-linked stock movements, invoice snapshots, separate payments, and controlled return/refund records.
 - Represent customer receivables and supplier payables accurately through unpaid, partially paid, and paid states.
 - Enforce the approved role-based access model using the existing Django authentication, Groups, Permissions, sessions, and server-side authorization checks.
 - Produce usable invoices/receipts and basic operational and financial reports from real stored transactions.
 - Extend the existing Django 6.1 modular-monolith architecture without replacing its authentication, UI foundation, app ownership, or database conventions.
-

1. Source of Truth and Precedence

The existing repository implementation is authoritative whenever this BRD conflicts with an older proposal or technical specification.

Precedence:

1. Existing repository implementation and current team decisions
2. This Phase 1 BRD
3. Older technical specification documents

No older document may silently reintroduce a feature explicitly deferred by this Phase 1 BRD.

2. Existing Foundation That Must Be Preserved

The project already exists and must be extended rather than replaced.

Phase 1 must preserve:

- Django 6.1
- Django Templates
- Django Forms / ModelForms
- Django ORM
- PostgreSQL hosted on Neon
- Tailwind CSS v4
- vanilla JavaScript
- `uv` for Python dependencies
- npm for frontend tooling
- feature apps under `apps/`
- shared layouts/components under root `templates/`
- the current shared UI foundation

The Pharmacist performs POS/cashier duties. There is no separate Cashier role.

Assumptions, Constraints & Non-Functional Requirements

Delivery and team constraints

- Phase 1 is delivered within 9 calendar days by a four-person team.
- The product is a single-pharmacy, full-stack Django modular monolith built by extending the existing repository rather than replacing its foundation.
- The delivery priority is the connected end-to-end Phase 1 workflow, not an exhaustive enterprise ERP.

Primary implementation ownership is:

Team member	Primary ownership
Mhmd Hajeer	Core/platform, authentication, shared design/UI, and final integration
Hala	Catalog, parties, inventory, purchasing, and supplier returns
Malik	POS, sales, prescriptions, and sales invoice/receipt output
Yasser	Finance, payments, customer returns/refunds, and reporting

Ownership coordinates implementation work; it does not change the business permissions in section 6.

Security

- Preserve the current username/password authentication built on Django's built-in `User`, `AuthenticationForm`, sessions, authentication middleware, and CSRF protection.
- Preserve the current login flow and POST-only, CSRF-protected logout flow; do not replace them with email login, JWT, DRF authentication, a custom User model, or a separate authorization system.

- Use Django Groups and Permissions with the exact groups `Owner / Admin`, `Pharmacist`, `Inventory Manager`, and `Accountant`.
- `Owner / Admin` receives full business-system access through group permissions; other access follows the matrix in section 6.
- Every protected view and business action enforces authentication and permission checks server-side. Preserve the current permission-aware navigation/context-processor architecture as a usability layer; it is never the security boundary.

Minimum audit and traceability

- The domain-specific actor, timestamp, source-reference, stock-movement, transaction-snapshot, and payment-reversal records defined in this BRD and the approved ERD are the required Phase 1 audit trail.
- Posted/completed financial and inventory history is retained; effective transactions are not silently rewritten or hard-deleted.
- Phase 1 does not require a generic `AuditEvent` model, enterprise audit platform, or complex reversal state machine.

Practical performance expectations

- Performance is scoped to normal small-scale, single-pharmacy Phase 1 use; no multi-branch or enterprise-load target is assumed.
- Common catalog lookup, barcode/POS lookup, FEFO allocation, invoice/payment lookup, and minimum reports use the approved database indexes, direct query/services, and targeted transaction locks without adding duplicate summary tables.
- Correct stock, financial, permission, and transaction behavior takes priority over caching or distributed infrastructure. Redis and formal enterprise performance targets are not required for Phase 1.

3. Phase 1 Goal

Phase 1 must provide one complete connected workflow:

```

Supplier
  ↓
Purchase Invoice / Receiving
  ↓
Medicine Batch / Inventory
  ↓
POS Sale
  ↓
Sales Invoice / Receipt
  ↓
Payment / Balance
  ↓
Return / Refund
  ↓
Basic Reports
  
```

4. Phase 1 Success Conditions

Phase 1 is complete when:

- medicines, suppliers, customers, and prescriptions can be managed;
 - purchases create/receive medicine batches and increase stock;
 - inventory is batch-aware and prevents selling expired stock;
 - sales reduce the correct inventory quantities;
 - FEFO is used for sale allocation;
 - sales produce a usable invoice/receipt;
 - payments are stored separately from invoices;
 - unpaid, partially paid, and paid balances are represented correctly;
 - customer and supplier returns affect the correct stock and financial balance;
 - basic reports use real stored transactions;
 - the system works against the shared Neon PostgreSQL database.
-

5. Phase 1 In-Scope Features

5.1 Authentication and Authorization

Authentication, session handling, authorization, exact group names, and the prohibition on a custom `Role` model are governed by the consolidated security requirements above. Role capabilities are defined in section 6.

5.2 Medicine Catalog

Manage:

- medicines;
- categories;
- manufacturers;
- medicine units;
- unit conversion to a base unit;
- barcodes;
- default selling price;
- prescription-required flag;
- low-stock threshold;
- active/inactive state.

Rules:

- each medicine has exactly one base unit;
- the base unit conversion factor is `1`;
- all conversion factors are greater than zero;
- barcodes are unique;
- medicine records with transaction history are deactivated instead of hard-deleted;
- posted sales/purchases preserve the unit and price/cost actually used.

Phase 1 unit economics

All unit economics use `Decimal`; float arithmetic is prohibited.

- `Medicine.default_selling_price` is the tax-exclusive selling price of one base unit.

- A selected quantity is expressed in the chosen `MedicineUnit`. Inventory and stock movements are always expressed in base units.
 - `base_quantity = selected_quantity × conversion_to_base`, quantized to `Decimal("0.001")` with `ROUND_HALF_UP`. The quantized result must be greater than zero.
 - For sales, `selected_unit_price = default_selling_price × conversion_to_base`, quantized to `Decimal("0.0001")` with `ROUND_HALF_UP`. `SalesInvoiceLine.unit_price` is this selected-unit price snapshot, while `quantity`, `conversion_to_base_snapshot`, and `requested_quantity_base` preserve the selected quantity, conversion, and quantized base quantity.
 - For purchases, `PurchaseInvoiceLine.unit_cost` is the tax-exclusive cost of one selected purchase unit. `received_quantity_base` is the quantized base quantity, and `acquisition_cost_per_base_unit = unit_cost / conversion_to_base_snapshot`, quantized to `Decimal("0.0001")` with `ROUND_HALF_UP`.
 - Phase 1 line discounts and tax affect invoice totals but do not alter the batch acquisition-cost layer; `MedicineBatch.acquisition_cost_per_base_unit` stores the selected-unit cost conversion above. Posted lines preserve all snapshots so later master-data changes do not alter history.
-

5.3 Suppliers, Customers, and Prescribers

Suppliers

Store at minimum:

- unique supplier code;
- name;
- contact person;
- phone;
- email;
- address;
- notes;
- active/inactive state.

Customers

Store at minimum:

- unique customer code;
- name;
- phone;
- email;
- address;
- notes;
- active/inactive state.

A sale may use a saved customer or a walk-in customer with no Customer row.

Prescribers

Store at minimum:

- name;
- optional phone;
- optional professional/reference identifier;
- notes;
- active/inactive state.

5.4 Inventory and Medicine Batches

Inventory is batch-aware.

A medicine may have multiple batches with different:

- batch numbers;
- expiry dates;
- acquisition costs;
- available quantities.

Required:

- medicine batches;
- current available quantity per batch;
- expiry tracking;
- low-stock detection;
- near-expiry detection;
- expired-stock visibility;
- append-style stock movement history.

Rules:

- expired batches can never be sold;
- normal workflows must not produce negative stock;
- stock-changing workflows must create a `StockMovement` ;
- batch quantity changes must happen through `apps.inventory` service functions;
- other apps must not independently mutate batch quantity;
- stock history must remain traceable to its source transaction.

The existing `MANUAL_ADJUSTMENT_IN` and `MANUAL_ADJUSTMENT_OUT` movement codes are reserved for a future approved workflow. Phase 1 provides no manual-adjustment page, endpoint, or general-purpose stock-editing service. They must not be used to bypass purchase, sale, or return workflows.

FEFO

Sales use **First Expired, First Out**.

Eligible batches:

1. belong to the selected medicine;
2. are active;
3. are not expired on the current UTC date, which is the repository's explicit Phase 1 business date;
4. have available quantity greater than zero.

Allocation order:

1. earliest expiry date;
2. earliest received/created batch when expiry dates tie.

A sale line may be allocated across multiple eligible batches.

5.5 Purchasing

Phase 1 purchasing is intentionally simple.

Required workflow:



Required purchase invoice information:

- unique internal invoice number;
- supplier;
- optional supplier invoice reference;
- invoice date;
- optional due date;
- status;
- subtotal;
- total discount;
- total tax;
- grand total;
- paid total;
- remaining balance;
- payment status.

Required purchase line information:

- medicine;
- purchase unit;
- quantity;
- conversion snapshot;
- unit cost;
- discount;
- tax;
- line total;
- batch number;
- expiry date.

Posting/receiving a purchase must:

- validate lines and batch data;
- create or identify the received batch;
- increase inventory through the inventory service;
- create stock movement history;
- preserve transaction snapshots;
- update the purchase invoice status.

Each posted `PurchaseInvoiceLine` maps to exactly one positive `PURCHASE_RECEIPT` `StockMovement` : `source_type = "PURCHASE_RECEIPT"` , `source_id = PurchaseInvoice.id` , and `source_line_id = PurchaseInvoiceLine.id` .

Explicit Phase 1 simplification

A separate Purchase Order and Goods Receipt workflow is not required in Phase 1.

5.6 Sales / Pharmacy POS

The Pharmacist uses the POS.

Required:

- medicine search;
- barcode search/scanning through common keyboard/HID scanners;
- unit selection;
- quantity entry;
- available-stock visibility;
- expiry warnings;
- multiple sale lines;
- saved customer selection;
- walk-in customer support;
- allowed discounts;
- tax calculation;
- optional prescription association;
- prescription-required warning;
- FEFO batch allocation;
- sale completion;
- printable invoice/receipt.

The server remains authoritative for:

- stock;
- unit conversions;
- FEFO;
- prices;
- discounts;
- taxes;
- totals;
- permissions;
- final transaction state.

Phase 1 prices are tax-exclusive. Owner/Admin and Pharmacist may apply a sales-line discount; Owner/Admin and Inventory Manager may apply a purchase-line discount. The server accepts a discount amount from zero through the rounded line subtotal. Phase 1 has no percentage-based approval tiers or additional discount-limit workflow.

Completing a sale must:

- validate permission and sale data;
- recalculate totals server-side;
- allocate non-expired batches using FEFO;
- create batch allocations;
- reduce stock through the inventory service;

- create stock movements;
- create/finalize the sales invoice;
- preserve price/cost/unit/tax/discount snapshots;
- record initial payment when supplied;
- calculate the remaining balance.

Each `SaleBatchAllocation` created during completion must map to exactly one negative `SALE` `StockMovement` in the same database transaction. For that movement, `source_type = "SALE"`, `source_id = SalesInvoice.id`, and `source_line_id = SaleBatchAllocation.id`; its batch must equal the allocation batch and `quantity_delta_base` must equal `- allocated_quantity_base`.

Phase 1 simplification

Held/resumed sales and physical-stock discrepancy overrides are deferred.

If recorded stock is insufficient, sale completion must fail with a clear validation message.

5.7 Prescriptions

Required:

- prescription record;
- optional customer;
- optional prescriber;
- prescription date;
- prescription lines;
- prescribed quantity where applicable;
- dosage/instructions;
- notes;
- optional attachment only if reasonably achievable without delaying core delivery.

A medicine marked `prescription_required` must show a warning during POS use.

The warning is non-clinical and does not provide medical advice.

Expired stock remains blocked regardless of prescription state.

5.8 Sales and Purchase Invoices

Invoices are historical transaction records.

Sales invoices must support:

- unique invoice number;
- optional customer;
- Pharmacist;
- sale date/time;
- lines;
- subtotal;
- discounts;
- tax;
- grand total;
- paid total;

- balance due;
- payment status.

Purchase invoices must support the equivalent supplier-side information.

Posted/completed invoices must preserve:

- medicine description snapshot;
- unit snapshot;
- conversion snapshot;
- price/cost snapshot;
- discount snapshot;
- tax snapshot;
- totals.

Completed transaction history should not be hard-deleted.

Output

Phase 1 requires clean, professional, print-friendly HTML invoices/receipts.

PDF generation is optional for Phase 1 and must not delay the core workflow.

5.9 Payments and Balances

Payments are separate from invoices.

Supported invoice states:

- unpaid;
- partially paid;
- paid.

Customer payments

Required:

- sales invoice;
- optional saved customer context;
- payment method;
- amount;
- optional reference;
- payment timestamp;
- processed-by user.

Supplier payments

Required:

- purchase invoice;
- supplier;
- payment method;
- amount;
- optional reference;
- payment timestamp;
- processed-by user.

Supported payment methods include:

- cash;
- card;
- bank transfer;
- other.

Rules:

- payments must be positive;
- multiple payments are allowed;
- payments cannot exceed the current outstanding invoice balance;
- posting/reversing a payment recalculates invoice paid total, balance, and payment status;
- walk-in sales must be fully settled at completion;
- saved customers may have unpaid/partial balances.

Customer and supplier payment reversals use the same minimal metadata: `status` changes from `POSTED` to `REVERSED`, `reversed_by` identifies the user, `reversed_at` records the timestamp, and `reversal_reason` may contain explanatory text. Reversal preserves the original payment and is performed by the finance service; it is not a generic reversal state machine.

Original invoice balances remain payment-only historical values:

- `SalesInvoice.paid_total` is the sum of its active `POSTED` customer payments, and `balance_due = grand_total - paid_total`;
- `PurchaseInvoice.paid_total` is the sum of its active `POSTED` supplier payments, and `remaining_balance = grand_total - paid_total`;
- returns, refunds, and supplier returns never rewrite invoice grand totals, line snapshots, paid totals, payment statuses, or invoice balances.

Customer and supplier statements are derived from invoices, active payments, returns, and refunds; no separate mutable balance table is required. Statements use the pharmacy's perspective: positive amounts are owed to the pharmacy and negative amounts are owed by the pharmacy. A sales invoice is positive, a customer payment is negative, a posted customer return credit is negative, and a customer refund paid is positive because it settles that credit. A purchase invoice is negative, a supplier payment is positive, and a posted supplier return is positive because it reduces the payable. Statement/net balances must apply each event once and must not be substituted for the original invoice balance fields.

5.10 Customer Returns and Refunds

A customer return must reference an original completed sale.

Required:

- original sales invoice;
- original sales line;
- exact originally allocated batch;
- returned quantity;
- reason;
- return condition;
- refund amount;
- processed-by user;
- timestamp.

Rules:

- cumulative returned quantity cannot exceed the quantity originally sold;

- each sales line/batch pair has at most one `SaleBatchAllocation`, so the original allocation is identified unambiguously by the original sales line and batch;
- a resellable, non-expired returned medicine may be restored to its original batch;
- damaged, unsafe, or expired medicine must not return to sellable inventory;
- inventory restoration must use the inventory service and create a stock movement;
- refund remains linked to the return and original invoice;
- refund cannot exceed the eligible refundable amount.

Each restocked `CustomerReturnLine` maps to exactly one positive `CUSTOMER_RETURN_RESTOCK` `StockMovement`: `source_type = "CUSTOMER_RETURN_RESTOCK"`, `source_id = CustomerReturn.id`, and `source_line_id = CustomerReturnLine.id`.

Customer refunds are posted-only in Phase 1. Reversing an already-posted refund is deferred because no refund-reversal workflow or metadata is approved; a refund must not be assigned a `REVERSED` state.

For Phase 1, the Pharmacist may classify a returned item as resellable/non-resellable as part of the permitted return workflow; Owner/Admin may also perform the action.

5.11 Supplier Returns

A supplier return must reference:

- supplier;
- related purchase invoice where available;
- medicine;
- exact batch;
- returned quantity;
- reason;
- return value.

Posting a supplier return must:

- reduce the exact batch through the inventory service;
- create stock movement history;
- preserve traceability;
- affect the supplier statement/net balance separately without rewriting the purchase invoice balance.

Each posted `SupplierReturnLine` maps to exactly one negative `SUPPLIER_RETURN` `StockMovement`: `source_type = "SUPPLIER_RETURN"`, `source_id = SupplierReturn.id`, and `source_line_id = SupplierReturnLine.id`.

5.12 Settings

Keep settings minimal.

Required:

- pharmacy name;
- phone/email/address;
- currency code;
- default tax rate;
- expiry-warning threshold;
- default low-stock threshold;
- payment methods;
- invoice/receipt header/footer information as needed.

The system is single-currency in Phase 1.

Posted documents snapshot their currency and relevant display information.

5.13 Dashboard and Reports

Use one dashboard architecture with role-aware content.

Minimum useful reports:

- sales;
- purchases;
- current inventory;
- low stock;
- near-expiry stock;
- expired stock;
- customer receivables;
- supplier payables;
- customer payments;
- supplier payments;
- basic gross profit / COGS where available.

Reports are query/service-based.

Do not create unnecessary report transaction models.

CSV export is optional if time permits.

XLSX/PDF report export is deferred unless core work is already complete.

6. Role and Permission Matrix

Django-generated model permissions should be used where they map cleanly to CRUD actions.

Custom permissions may be created for non-CRUD business actions such as:

- `sales.complete_sale`
- `purchasing.post_purchaseinvoice`
- `finance.post_customerpayment`
- `finance.post_supplierpayment`
- `returns.post_customerreturn`
- `returns.post_supplierreturn`
- `returns.process_refund`
- `finance.view_financial_reports`

`finance.view_financial_reports` is declared on `CustomerPayment`, not on a fake reports model. It is assigned only to Owner / Admin and Accountant. Pharmacist and Inventory Manager use owning-app permissions for their scoped operational and inventory/purchasing reports.

Suggested Phase 1 capability matrix:

Capability	Owner / Admin	Pharmacist	Inventory Manager	Accountant
Dashboard	Full	Yes	Yes	Yes
User/group/permission administration	Full	No	No	No
Medicine catalog	Full	View/search	Full	View
Batch/inventory lookup	Full	View	Full	View
Stock changes	Full	Through sale/return only	Through purchase receiving and supplier returns only; no direct/manual adjustment workflow in Phase 1.	No
Suppliers	Full	No	Full	View
Customers	Full	Create/view/select	View	View
Prescriptions	Full	Full	No	No
Purchases	Full	No	Full	View
POS/sales	Full	Full	View only if needed	View
Customer payments	Full	Permitted POS/customer payments	No	Full
Supplier payments	Full	No	No	Full
Customer returns/refunds	Full	Operational access	Inventory visibility	Financial visibility
Supplier returns	Full	No	Full	Financial visibility
Reports	Full	Operational through owning-app permissions	Inventory/purchasing through owning-app permissions	Financial through <code>finance.view_financial_reports</code>
Settings	Full	No	No	View only if required

7. Phase 1 Django App Boundaries

Recommended structure:

```
apps/
├─ accounts/      # existing auth integration
├─ dashboard/     # existing dashboard shell
├─ core/          # minimal settings, tax rates, payment methods
├─ catalog/       # medicines, categories, manufacturers, units, barcodes
├─ parties/       # suppliers, customers, prescribers
├─ inventory/     # batches, stock movements, inventory services
├─ purchasing/    # purchase invoices and receiving
├─ sales/         # POS, sales invoices, batch allocations
├─ prescriptions/ # prescriptions and lines
├─ finance/       # customer/supplier payments and balances
├─ returns/       # customer/supplier returns and refunds
└─ reports/       # query/report services; no transaction models
```

Each business model has one authoritative owning app.

Other apps may use its public services/queries but must not duplicate its data model or business rule.

The stable navigation labels map to the implemented owning-app namespaces as follows:

Navigation label	Namespace / owning app
Dashboard	<code>dashboard</code>
Sales	<code>sales</code>
Medicines	<code>catalog</code>
Inventory	<code>inventory</code>
Suppliers	<code>parties</code>
Customers	<code>parties</code>
Prescriptions	<code>prescriptions</code>
Purchases	<code>purchasing</code>
Invoices	<code>sales</code>
Payments	<code>finance</code>
Returns & Refunds	<code>returns</code>
Reports	<code>reports</code>
Settings	<code>core</code>

Labels such as Medicines, Invoices, Payments, and Settings are presentation concepts; they do not authorize duplicate Django apps with those names.

8. Transaction and Service Rules

Simple CRUD may use normal Django views/forms/models.

Use explicit service functions for multi-record business operations such as:

- posting/receiving a purchase;
- completing a sale;
- FEFO allocation;
- posting a customer payment;
- posting a supplier payment;
- processing a customer return/refund;
- processing a supplier return.

Use `transaction.atomic()` where several related writes must succeed or fail together.

`transaction.atomic()` must be combined with targeted `select_for_update()` locking for the rows whose current values determine whether a transaction may post:

- purchase posting locks the purchase invoice and any existing batch cost-layer row that will be increased;
- sale completion locks the sales invoice and eligible `MedicineBatch` rows in deterministic FEFO order (`expiry_date` , `first_received_at` , `id`) before availability is revalidated and decremented;

- customer and supplier return posting locks the return plus every affected batch before quantity validation and mutation;
- customer/supplier payment posting or reversal locks the affected invoice before its outstanding balance is checked and recalculated.

Services must re-check status, stock, and balance after acquiring locks. This is the minimum concurrency protection for Phase 1; a generic locking framework is not required.

For Phase 1 transaction statuses, `VOID` means only that an unposted/uncompleted `DRAFT` was cancelled and retained for traceability. A void draft has no stock movements, batch allocations, payments, refunds, or balance effect. Phase 1 services may allow only `DRAFT → VOID`; they must not expose `POSTED → VOID` or `COMPLETED → VOID`. Reversing an effective transaction requires compensating inventory and financial behavior and remains deferred.

The most important ownership rule is:

`apps.inventory` is the only app allowed to directly change `MedicineBatch.quantity_available_base`.

Every increase/decrease must create a corresponding `StockMovement` in the same transaction.

9. Basic Data Rules

- New project-owned business entities use UUID primary keys.
- Existing Django auth tables keep Django's normal IDs.
- Quantities use Decimal fields with three decimal places.
- Unit conversion factors use Decimal fields with six decimal places.
- Unit prices/costs use Decimal fields with four decimal places.
- Posted totals/payments use Decimal fields with two decimal places.
- Tax rates use Decimal fields.
- Do not use floating-point arithmetic for money.
- Timestamps are timezone-aware and stored by Django in UTC (`USE_TZ = True`).
- The repository's explicit Phase 1 business timezone is UTC (`TIME_ZONE = "UTC"`), so FEFO expiry eligibility uses `timezone.localtime()` under UTC. Changing to another pharmacy timezone later requires an explicit settings/code decision and regression tests around midnight; agents must not infer a timezone from a developer machine.
- Master data with history is deactivated rather than hard-deleted.
- Payments remain separate from invoices.

Document numbering

Phase 1 does not use a mutable sequence table. Services generate concurrency-safe, human-readable identifiers deterministically from the record's UUID, using the complete uppercase 32-character UUID hex value:

- sales invoice: `SAL-{uuid_hex}`;
- purchase invoice: `PUR-{uuid_hex}`;
- customer return: `CRT-{uuid_hex}`;
- supplier return: `SRT-{uuid_hex}`;
- customer refund: `CRF-{uuid_hex}`.

All formats fit the existing `CharField(max_length=40)` fields. Draft sales and purchase invoices may keep an empty number until completion/posting; the service assigns the deterministic number before changing status. Return/refund numbers are assigned when their records are created. Database uniqueness constraints remain mandatory. User-entered internal document numbers and sequential numbering are not part of Phase 1; supplier-provided invoice references remain separate.

9.1 Financial calculation and rounding policy

All authoritative calculations use `Decimal`; float arithmetic is prohibited. Prices/costs are tax-exclusive, and tax is calculated after the line discount. Sales lines use:

```
money_quantum = Decimal("0.01")
line_subtotal = (quantity * unit_price).quantize(
    money_quantum,
    rounding=ROUND_HALF_UP,
)
taxable_amount = line_subtotal - discount_amount
tax_amount = (
    taxable_amount * tax_rate_percent / Decimal("100")
).quantize(money_quantum, rounding=ROUND_HALF_UP)
line_total = (taxable_amount + tax_amount).quantize(
    money_quantum,
    rounding=ROUND_HALF_UP,
)
```

Purchase lines use the same sequence with `unit_cost` in place of `unit_price`. Quantity, conversion, and four-decimal unit price/cost values retain their approved precision until multiplication. The line subtotal is rounded to two decimals before applying the stored two-decimal discount; the discount must not exceed that subtotal. Tax is then rounded once to two decimals, followed by the line total.

Invoice subtotal, discount, tax, and grand total are sums of the already-rounded line snapshots and are stored at two decimals. Payments, refunds, paid totals, and balances use two decimals and `ROUND_HALF_UP` for any required quantization. Reports sum stored posted monetary values rather than recalculating historical tax. For COGS, allocation quantity × acquisition-cost snapshot is calculated at full Decimal precision, summed per sales line, then quantized to two decimals with `ROUND_HALF_UP` before report aggregation.

10. Explicitly Deferred to Phase 2+

The following are **not required for Phase 1**:

- purchase orders;
- separate goods-receipt workflow;
- held/resumed sales;
- cashier/pharmacist shift reconciliation;
- physical-stock discrepancy override/investigation;
- stock count workflow;
- advanced stock reconciliation;
- generic idempotency infrastructure;
- enterprise audit-event infrastructure;
- document-sequence infrastructure;
- complex reversal state machines;
- complex row-locking architecture everywhere (the targeted locks required by section 8 are not deferred);
- Redis;
- JWT;
- DRF;
- React/Vue/Angular SPA;

- multi-branch support;
- full double-entry accounting/general ledger;
- multi-currency;
- enterprise approval workflows;
- XLSX export as a mandatory requirement;
- PDF reports as a mandatory requirement;
- e-commerce;
- delivery;
- loyalty;
- payroll/HR;
- insurance integrations;
- AI medical recommendations or clinical decision-making.

A deferred feature may only be added after the team explicitly approves it and core Phase 1 is safe.

11. Backup and Recovery

For Phase 1:

- document how the shared Neon/PostgreSQL development database can be backed up/exported;
- keep credentials outside Git;
- do not build a custom backup subsystem.

A formal automated backup/restore feature is not required.

12. Testing Priorities

At minimum test:

- anonymous access denial;
- allowed/denied permissions by group;
- medicine units/conversions/barcodes;
- batch expiry blocking;
- purchase posting increases stock;
- stock movements are created;
- FEFO allocation;
- multi-batch sale allocation;
- insufficient stock rejection;
- sale completion reduces stock;
- walk-in sale settlement;
- partial/full saved-customer payments;
- supplier payments;
- customer return quantity limits;
- safe return restocking;
- unsafe/expired return non-restocking;
- supplier return stock reduction;

- invoice totals and snapshots;
 - basic report queries.
-

13. Implementation Gate

This BRD defines the approved minimum Phase 1 business scope.

`docs/ERD.md` is the approved Phase 1 schema baseline. Baseline business models and migrations exist in the repository as of commit `5ce85db`. Future schema changes must preserve documented app ownership, be reviewed against the approved BRD/ERD, and use new migrations rather than rewriting shared migration history.

No agent may silently reintroduce deferred Phase 2 features.